

Knowing Where to Look Is Not Knowing What Is Right: A Difference-Aware Transformer for Physical Commonsense QA

Group 69

Abstract

PIQA asks which of two near-identical solutions achieves an everyday physical goal. We design DACT, a Difference-Aware Contrastive Transformer trained from scratch, which marks the tokens where the two solutions differ, compares the solutions with cross-solution attention, pools with difference-guided attention and adds a jointly trained lexical head. With 1.9M parameters and no external data, DACT reaches 61.1% validation and 61.6% test accuracy (3 seeds), the best of our from-scratch models and above a TF-IDF baseline on both splits, although the test gap to that baseline is not significant. Controlled ablations show that difference tags, the lexical head and the pairwise objective each contribute, and attention controls show that the focus on differing tokens is learned rather than built in. The model finds the decisive words but cannot judge them: a fine-tuned RoBERTa is 6.3 points and a zero-shot 1.5 B LLM 13.5 points better. We report the development history, including the failures that shaped the final design.

1 Introduction

Physical commonsense, knowing that *baby wipes* rather than *index cards* remove chalk dust, is easy for people and hard for machines. PIQA (Bisk et al., 2020) tests it with a goal and two candidate solutions, exactly one of which is physically sensible. The task matters for assistants that give practical instructions and for robots that must choose feasible actions. Its key difficulty is that the two solutions are usually near-duplicates: in our training split the median share of differing solution tokens is 15%, so the answer hinges on a few substituted words whose plausibility requires world knowledge.

Our aim was a model that is trained from scratch but built around this structure. DACT (Section 2)

makes the difference between the solutions explicit at every stage: in the input, in the comparison and in the summary vector, and it adds a lexical head that scores which words make a solution plausible. We compare it with five baselines, from majority vote to a zero-shot LLM, run eight ablations with three seeds each, and analyse the attention with controls and probes.

Our main findings are: (i) DACT is the best from-scratch model and at least as good as a strong lexical baseline; (ii) its attention concentrates on the differing words, and controls show this focus is learned; (iii) errors come from *judging* those words without physical knowledge, which only pretraining supplies. Section 5 documents what worked and what did not during development.

2 Methodology

Input and difference tags. For a goal g and solutions s_1, s_2 , each option k is encoded as $x^{(k)} = [\text{CLS}] g [\text{SEP}] s_k [\text{SEP}]$ with a BPE vocabulary (Sennrich et al., 2016) learned on the training split. We align the token sequences of s_1 and s_2 with `diff1ib` and give every solution token a tag $t_i \in \{\text{shared}, \text{different}\}$ (goal and special tokens: *other*). The input embedding is

$$e_i = E_{\text{tok}}[x_i] + E_{\text{pos}}[i] + E_{\text{seg}}[\sigma_i] + E_{\text{tag}}[t_i]. \quad (1)$$

The tag embedding (our contribution) tells the encoder from the first layer where the options disagree.

Encoder. A shared pre-LayerNorm Transformer encoder (Vaswani et al., 2017; Xiong et al., 2020) maps each option to hidden states $H^{(k)} \in \mathbb{R}^{L \times d}$.

Contrastive cross-solution attention. Each token of option k attends to the solution tokens of the rival option $\bar{k}$ plus its CLS token as a “no counterpart” sink:

$$c_i = \text{MHA}(h_i^{(k)}, H^{(\bar{k})}, H^{(\bar{k})}), \quad (2)$$

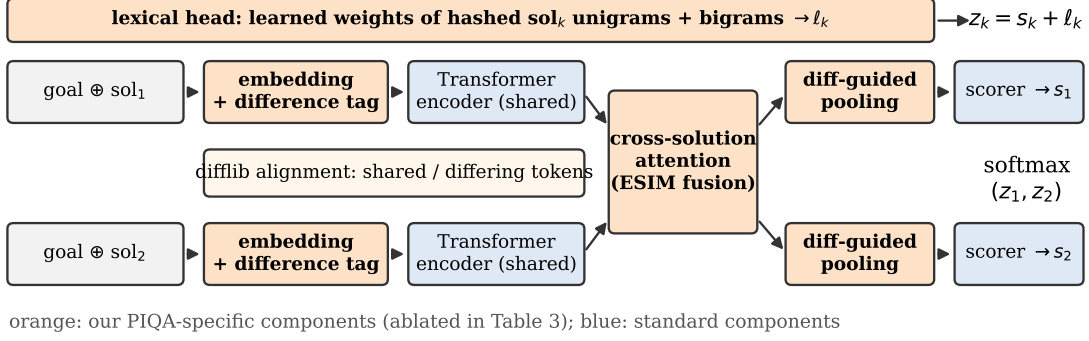


Figure 1: DACT. Both options pass through the same weights, so the model is permutation-equivariant (swapping the options swaps the scores; verified in the notebook).

and the states are updated with the comparison features of ESIM (Chen et al., 2017): $\tilde{h}_i = \text{LN}(h_i + \text{FFN}([h_i; c_i; h_i - c_i; h_i \odot c_i]))$. Adapting ESIM’s local inference from premise–hypothesis pairs to two competing answers is our design choice: plausibility is relative, so each option is represented by what it has *instead* of its rival.

Difference-guided pooling. Over the solution tokens we use additive attention (Bahdanau et al., 2015) with a learned bias per tag (our extension):

$$\alpha_i \propto \exp(v^\top \tanh(W\tilde{h}_i) + b_{t_i}), \quad p = \sum_i \alpha_i \tilde{h}_i, \quad (3)$$

with $b_{\text{different}}$ initialised to 1 and the others to 0. A scorer MLP maps $[p; \tilde{h}_{\text{CLS}}]$ to a scalar s_k .

Lexical head. Following the wide & deep idea (Cheng et al., 2016), a sparse linear component scores the words of each solution directly: every BPE unigram and bigram of s_k is hashed into one of 2^{18} buckets with learned weight w_j (initialised to 0),

$$\ell_k = \sum_{j \in \text{ngrams}(s_k)} w_{h(j)}, \quad z_k = s_k + \ell_k. \quad (4)$$

It is trained jointly with the Transformer, with its own learning rate and weight decay. Under the pairwise softmax only $\ell_1 - \ell_2$ matters, so shared n-grams cancel and the head learns which substituted words indicate a plausible solution, the signal that a TF-IDF baseline exploits.

Objective. The two scores compete in a softmax, $P(k) = \text{softmax}(z_1, z_2)_k$, trained with cross-entropy and label smoothing 0.1, which matches the forced-choice task. Every component can be switched off in a single configuration object, which is how the ablations are run.

3 Experimental Setup

Data. PIQA provides 16,113 training items and 1,838 labelled development items; the latter serve as our test set and are used only in the final evaluation. We hold out a stratified 10% of the training file (seed 42) for validation: 14,501 / 1,612 / 1,838 items, all label-balanced (50.0 / 50.0 / 50.5% label 1). Text is whitespace-normalised, NFKC-normalised and lower-cased. Goals are truncated to 40 and solutions to 96 BPE tokens; for longer solutions (1.6% of test solutions) we keep the 96-token window centred on the differing span, not the prefix (Section 5).

Training. AdamW (Loshchilov and Hutter, 2019) ($\beta_2=0.98$, weight decay 0.05, learning rate 3×10^{-4} ; lexical head 10^{-2}), 6% linear warm-up then cosine decay, batch 128, gradient clipping 1.0, dropout 0.2, at most 20 epochs with early stopping on validation accuracy (patience 6), fp16 mixed precision on a Colab T4. Step 1 of the notebook selects the configuration on validation with one seed from five candidates: base ($d=256$, 4 layers) 61.2%, **small** ($d=128$, **2 layers, 4 heads**) **61.4%**, base with dropout 0.3 60.8%, and base/small with a 10-epoch in-domain masked-LM warm-up (Devlin et al., 2019) 60.8 / 60.9%. The selected model has 1.93 M parameters, 262 k of them in the lexical head. The lexical head’s learning rate was chosen in a separate validation pilot (Section 5).

Baselines. All baseline numbers come from our own runs. **B0** majority class. **B1** TF-IDF of 1–2-grams of $s_2 - s_1$ with logistic regression, C tuned on validation; it tests how far shallow lexical cues go (Gururangan et al., 2018). **B2** a 2-layer

Model	Val	Test
<i>trained from scratch</i>		
B0 majority	50.0	50.5
B1 TF-IDF + LR	59.2	61.2
B2 BiLSTM + attention	59.8 ± 0.3	58.7 ± 0.9
DACT (ours)	61.1 ± 0.8	61.6 ± 0.7
<i>pretrained</i>		
B3 RoBERTa-base, fine-tuned	69.0 ± 0.5	67.9 ± 0.8
B4 Qwen2.5-1.5B, zero-shot	78.2	75.1

Table 1: Accuracy (%), mean $\pm$ std over 3 seeds; B0, B1 and B4 are deterministic single runs. DACT test 95% CI: 59.2–63.6.

BiLSTM with additive attention and the same tokenizer (5.1 M parameters), a recurrent model of similar budget. **B3** roberta-base (Liu et al., 2019) fine-tuned with a multiple-choice head (4 epochs, learning rate 2×10^{-5} , 3 seeds, best epoch on validation), representing pretrained encoders. **B4** Qwen2.5-1.5B (Qwen Team, 2024) zero-shot: the prompt “Goal: g / Solution:” followed by each solution, choosing the higher length-normalised log-likelihood, which measures LLM knowledge without any training.

Metrics. Accuracy is PIQA’s official metric; labels are balanced and each item has exactly one correct option, so it is unbiased with 50% chance level. Trained models are run with seeds 42–44 and reported as mean $\pm$ std. For the seed with the best *validation* accuracy we give a 95% bootstrap CI (2,000 resamples; Efron and Tibshirani, 1993) and an exact McNemar test (McNemar, 1947) against DACT on the test set.

4 Results and Analysis

Overall comparison. DACT is the best from-scratch model on both splits (Table 1): +1.3 / +2.9 points over the BiLSTM (McNemar $p=0.021$ on test) and +1.9 / +0.4 over TF-IDF. The test gap to TF-IDF is small and not significant ($p=0.88$), so our claim is that DACT is at least as good as the lexical baseline, not that it clearly beats it. Pre-training dominates: RoBERTa is 6.3 and the zero-shot LLM 13.5 points above DACT on test, in line with PIQA being designed to test knowledge rather than reading skill (Bisk et al., 2020).

Ablations. Each ablation tests one hypothesis about a design choice (Table 2); everything else, including seeds, data and budget, is fixed. *Marking the difference helps*: removing the tags costs

Ablation (vs. full DACT)	Δ Val	Δ Test
– difference tags	–1.4	–1.3
– lexical head	–1.9	–0.9
pointwise (independent BCE) objective	–1.0	–0.8
mean instead of attention pooling	–0.2	–0.9
– cross-solution attention	–0.1	–0.7
– tag bias in pooling	+0.3	0.0
tag bias initialised at 0	+0.3	0.0
vanilla (all of the above removed)	–2.4	–3.0

Table 2: Ablations, 3 seeds each, one change at a time on the selected configuration; mean accuracy differences in points (seed std $\approx$ 0.3–0.9).

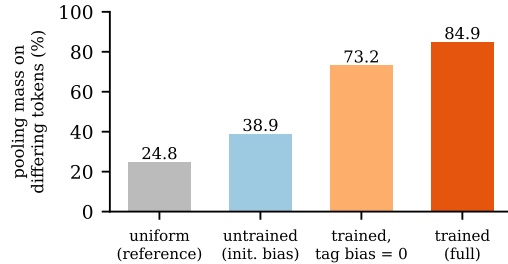


Figure 2: Share of pooling attention on differing tokens (test set). Removing the tag bias from the trained model keeps most of the focus, so it is learned, not built in.

1.4 / 1.3 points. *Direct lexical evidence helps*: removing the lexical head costs 1.9 / 0.9. *The forced-choice objective helps*: independent binary scoring costs 1.0 / 0.8. Removing everything costs 2.4 / 3.0 points, the largest drop. Cross-solution attention and attention pooling make no difference on validation and small ones on test, and the tag bias, including its initial value, makes none: these components overlap with the tags, which already tell the model where the options differ. Differences below one seed std are not conclusive, and with seven comparisons single p-values should not be over-read (Dror et al., 2018).

Complementary errors. DACT and TF-IDF agree on 78.8% of test items; each solves about 10.6% the other misses (197 items only DACT), for an oracle accuracy of 71.9%. The items only DACT solves typically have a one-word difference inside long shared text (*water* vs. *soda*; the typo *sparrow bottle* vs. *spray bottle*), where a bag-of-words difference vector has almost nothing to work with. Against RoBERTa the oracle rises to 83.0%.

Is the attention focus learned? The trained model puts 84.9% of its pooling attention on dif-

fering tokens, which make up 24.8% of solution tokens (Figure 2). Because the tag bias starts at 1, we separate the sources: an untrained model reaches only 38.9%, while the trained model with the bias set to 0 still reaches 73.2%, and the bias itself barely moved in training ($1.0 \rightarrow 1.022$). The focus is therefore learned by the pooling scorer, which also explains why initialising the bias at 0 changes nothing. Attention weights are not a complete explanation of a decision (Jain and Wallace, 2019; Wiegrefe and Pinter, 2019), so we use them as evidence of where the model looks, not of why it decides.

Why does the neural part alone not beat bag-of-words? We tested two explanations on validation. *H1: the pooling does not select* is rejected: the normalised pooling entropy is 0.51 (1 = uniform), for correct and wrong predictions alike ($p=0.12$). *H2: the difference signal is missing*: a linear probe separating differing from shared tokens reaches 59.6% balanced accuracy on the vanilla Transformer’s embeddings and only 60.4% after its encoder, so without tags the encoder barely works out where the options differ, whereas in DACT the tag remains perfectly decodable (100%) in every layer. Tags and the lexical head thus supply information the encoder does not learn from 14.5 k items. Crucially, wrong predictions put *more* attention on the differing tokens than correct ones (86.7 vs. 83.8%, $p<0.001$): looking at the right words is not the bottleneck; judging them is.

Qualitative analysis. For the item *adding scents to lotion bars* (Figure 3, top), the wrong option adds the oils “to the water in the bottom of the double boiler”. The pooling peaks exactly on *the water in the bottom*, the part that makes the option wrong, yet the model prefers it with $p = 0.99$: it locates the decisive span but cannot know that oil in the water bath never reaches the lotion. In *how to pry pallet apart* (bottom), pooling selects the decisive spans *pry apart / other end of* vs. *press / hammer down on*, but the scores tie although the gold option repeats the goal words *pry* and *apart*: DACT has no explicit goal–solution matching term. Successes rest on the same word-level evidence: for *a simple camping lantern*, the gold option (a milk jug on top of an *LED light, translucent plastic*) wins with $p = 1.00$ against *fairy lights* in a filled jug, with pooling spread over the many differing words. In an earlier run, *baby wipes* vs. *in-*

dex cards for chalk dust was solved confidently; the training split contains *baby wipe(s)* only in the correct option 26 times and only in the wrong option 5 times, so the confidence comes from word statistics, not from reasoning about absorbency. Finally, all four most confident predictions of the final model ($p \geq 0.99$, right or wrong) are pairs that differ in most of their words; since ℓ_k is a sum over n-grams, long dissimilar options receive extreme scores, a calibration side effect of the lexical head.

5 What Worked and What Did Not

The final model is the result of three complete runs (and two aborted ones); all are logged in the repository, and only the last one is used in Tables 1–2.

Run 1 (A100; no lexical head, prefix truncation). DACT scored 60.4% on test, *below* TF-IDF (61.2%), most single-component ablations scored higher than the full model on test, and the reported “64% attention on differing tokens” could have been an artefact of the bias initialisation. One item had its only difference at BPE positions 97–98, past the 96-token limit, so both inputs were identical after truncation ($p = 0.50$ exactly).

Fixes that worked. Difference-aware truncation keeps the differing span; in a unit check on long synthetic items it removed every case of identical inputs, and the affected PIQA item is no longer among the most uncertain predictions. The attention controls of Figure 2 turned the initialisation worry into a testable question, and the answer was that the focus is learned. Running RoBERTa with three seeds exposed its instability: in run 2 one seed stayed at chance (53.7% validation, loss ≈ 0.694 for all epochs), a known fine-tuning failure (Dodge et al., 2020), while in the final run all seeds trained (68.4–69.4%).

Engineering failures. On the free T4, PyTorch reported bf16 as supported although it is only emulated; training ran at 377 ms per step instead of 108 ms with fp16, and two free-tier sessions were reclaimed after 35–40 minutes. Using bf16 only with native support and keeping a browser tab attached to the session solved both.

Closing the gap to bag-of-words. Run 2 (T4, no lexical head) confirmed the picture of run 1: DACT 59.4 / 60.2% vs. TF-IDF 59.2 / 61.2% (val / test). We then added the lexical head. Its first version, trained with the Transformer’s learning rate,

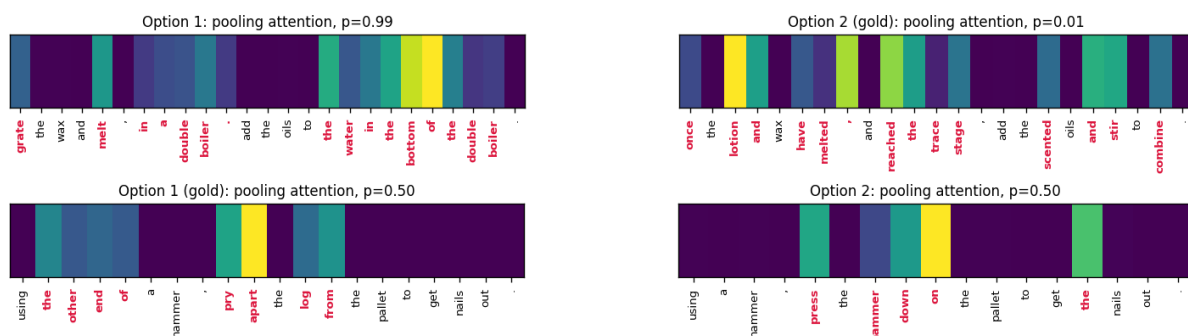


Figure 3: Pooling attention (bright = high; red = differing tokens). Top: *adding scents to lotion bars*, a confident error ($p = 0.99$ for option 1). Bottom: *how to pry pallet apart*, the most uncertain item ($p = 0.50$; option 1 is gold).

did not help in a validation pilot ($59.4 \rightarrow 59.5\%$) because its weights barely left zero before early stopping; a separate learning rate did ($3e-3$: 60.5, **1e-2**: **60.9**, $3e-2$: 60.6%; 3 seeds, test split not loaded). The final run confirmed the gain (61.1 / 61.6%). *Protocol note*: the head’s configuration was selected on validation only, but the decision to explore a lexical component was taken after we had seen the test results of runs 1 and 2. We therefore treat the small test gap to TF-IDF as indicative rather than as a clean held-out result.

Evidence. The submitted notebook contains the complete execution of the final code (commit 538cf5b, 57 cells, no errors) with the training logs of all 30 runs; outputs/ holds 40 JSON-lines logs, the result tables and figures; a final notebook cell asserts that tables, result files and logs agree; and docs/DEVELOPMENT_LOG.md records every step, including the failed attempts above.

6 Conclusion

DACT shows that task-specific structure pays off even when training from scratch: making the difference between two near-identical solutions explicit and adding direct lexical evidence yield the best from-scratch model, and every major component is supported by a controlled ablation. Its attention demonstrably learns to focus on the decisive words. The analysis equally shows the limit of this approach: the model finds *where* the options differ but judges *which* is right with word statistics from 14.5 k items, leaving it 6–14 points behind pretrained models.

Limitations. Improvements of about one point are within seed and test-sample noise; hyperparameters were selected with one seed; the lexical head is over-confident on dissimilar options;

and the test comparison with TF-IDF is not fully clean (Section 5). Natural next steps are an explicit goal–solution matching term, length-normalised lexical scores, and combining the difference-aware design with pretrained representations.

Team Contributions

[To be completed by the group: each member’s contribution to design, implementation, experiments, analysis and writing.]

Use of AI tools. An AI coding assistant (Claude Code) was used extensively in this project, including for implementation, experiments and drafting this report; the full record is in docs/DEVELOPMENT_LOG.md. *[The group should confirm and complete this statement according to the unit’s policy on AI use.]*

References

- Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2015. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations*.
- Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. 2020. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 7432–7439.
- Qian Chen, Xiaodan Zhu, Zhen-Hua Ling, Si Wei, Hui Jiang, and Diana Inkpen. 2017. Enhanced LSTM for natural language inference. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1657–1668.
- Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, and Hemal Shah. 2016. Wide & deep

- learning for recommender systems. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*, pages 7–10.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186.
- Jesse Dodge, Gabriel Ilharco, Roy Schwartz, Ali Farhadi, Hannaneh Hajishirzi, and Noah A. Smith. 2020. Fine-tuning pretrained language models: Weight initializations, data orders, and early stopping. *arXiv preprint arXiv:2002.06305*.
- Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. 2018. The hitchhiker’s guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1383–1392.
- Bradley Efron and Robert J. Tibshirani. 1993. *An Introduction to the Bootstrap*. Chapman and Hall/CRC.
- Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel Bowman, and Noah A. Smith. 2018. Annotation artifacts in natural language inference data. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 107–112.
- Sarthak Jain and Byron C. Wallace. 2019. Attention is not explanation. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3543–3556.
- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*.
- Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *International Conference on Learning Representations*.
- Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157.
- Qwen Team. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.
- Sarah Wiegrefe and Yuval Pinter. 2019. Attention is not not explanation. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 11–20.
- Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. 2020. On layer normalization in the transformer architecture. In *Proceedings of the 37th International Conference on Machine Learning*, pages 10524–10533.